

# Efficient Thinking: Tradeoffs in Game-Playing AI

---

## Parameters vs Search vs Self-Learning — a Three-Stage Chess Study of Where Strength Comes From

---

Louay Alsakka · July 8, 2026

A white paper on parameter-efficient game AI.

Code & trained model: [github.com/louayalsakka/efficient-thinking](https://github.com/louayalsakka/efficient-thinking)

*The recurring theme is a set of tradeoffs: memory vs compute (Stage 1 vs 2), speed vs score (the search cascade), and diversity vs correlation (the committee) — each one a different way of spending a fixed budget to buy strength, and each capped by the same wall: the quality of the learned evaluator.*

---

### Abstract

---

We study how playing strength in chess arises from three separable sources — **network capacity, search, and self-learning** — using a deliberately tiny model on modest hardware. A **3.45-million-parameter (14 MB) convolutional network** plays at **~2150 Elo** as a single-pass policy; adding **Monte-Carlo Tree Search (MCTS)** lifts the *same weights* to **~2800 Elo (top-human)** with **zero extra parameters** — strength bought entirely with compute. We make three further contributions. (1) A direct **MCTS-vs-fixed-depth** comparison: adaptive MCTS both **beats** alpha-beta at equal compute **and keeps scaling**, while fixed-depth search plateaus. (2) A **search-efficiency** result: an all-MCTS **wide→narrow cascade** that funnels the simulation budget through progressively narrower, deeper stages **matches flat MCTS strength while running up to ~4.8× faster per move** — the same answer for far less compute. (3) A **teacher-free self-learning** study (self-play, a self-referential ladder, evolution, and committees) with one robust positive and sober negatives: **model agreement predicts correctness** (a confidence signal needing no oracle), but *none* of these methods — including outcome-based evolution — crosses the self-play plateau, and plurality voting does not reliably de-bias. Since the same net reaches higher under supervised labels, the wall is the quality of *self-generated* signal, not capacity. Stage 3's constraint — **no engine, no labels** — is deliberate: it is the setting of any *genuinely new* problem, where no teacher or dataset exists to imitate, so a recipe that depends on them (as

Stages 1–2 do) cannot transfer; only a self-bootstrapping method can. Throughout, the unifying finding is that **the ceiling is the evaluator (the network), not the loop around it** — search and self-play *redistribute* the knowledge in the net; they do not manufacture it.

**Headline:** *A 14 MB model reaches ~2800 Elo by thinking (search), not by growing (parameters) — and once the search is good, the only wall left is the quality of the evaluation.*

---

## 1. Introduction

---

Modern game AI conflates three separable questions: 1. **Open loop** — how strong is a single forward pass (pure "intuition")? 2. **Closed loop** — how much does *search* (lookahead on a learned value function) add? 3. **Self-learning** — can the system improve *itself*, with no external teacher?

We treat these as three stages of increasing autonomy and measure each independently. The framing is explicitly **control-theoretic** (Bertsekas 2022, *Lessons from AlphaZero for Optimal, Model Predictive, and Adaptive Control*): the open-loop policy is a **feedforward controller**, closed-loop search is **model-predictive control** (receding-horizon planning with a learned terminal cost), and self-play is **iterative/adaptive learning control**. Chess is only a testbed; the goal is a *generic* recipe for sequential decision-making, and this study quantifies what each ingredient buys.

**Contributions.** - A parameter-efficiency result: **~2800 Elo from 3.45M params** via search, at constant memory. - A direct **MCTS-vs-fixed-depth** result: adaptive search beats and out-scales fixed depth. - A **search-efficiency** result: a wide→narrow MCTS **cascade** matches flat MCTS at up to **4.8× less compute per move**, with a clean score/speed trade-off curve. - An **architecture-beats-scale** finding (convolution » MLP at equal data), with topology sweep. - A reproducible **negative result** on small-scale self-play (plateaus below supervision). - A **teacher-free self-learning** study: agreement is a validated **confidence meter**, but self-play, a self-referential ladder, **evolution**, and plurality-voting committees all fail to cross the plateau — a set of clean negative results with a methodological warning (noisy fitness manufactures phantom gains).

---

## 2. Problem Setup and Methods

---

### 2.1 Representation

- **Position** → **move** as a 4096-way ( $64 \times 64$  from-to) classification. Promotions default to queen.
- **Side-to-move normalization:** the board is always presented from the mover's perspective (mirror + color-swap for Black), so one network serves both players.
- **Encodings:** *onehot* =  $64 \text{ squares} \times 12 \text{ piece planes} + 5 \text{ meta bits} = 773 \text{ floats}$  (used throughout). For convolutions the 773-vector reshapes to  $8 \times 8 \times 12 \text{ piece planes} + 5 \text{ meta planes} = 17 \text{ input channels}$ .

### 2.2 Labels and objective

- **Supervised data:** the full public Lichess cloud-eval database — **394,669,566 positions** (79 shards), each with Stockfish multi-PV win-probabilities.
- **Hard objective:** cross-entropy to the single best move (bounded by imitation).
- **Soft objective:** cross-entropy to an *advantage-weighted distribution* over legal moves ( $\text{softmax}(v_i/\tau)$ ). Soft is not bounded by copying one move and generalizes better.

### 2.3 Evaluation

- **Elo via a Stockfish ladder** (random baseline, then SF at set UCI\_Elo rungs), 20–80 games/rung; Elo fit by maximum-likelihood against the known rung ratings.
- **Methodological caveat (important):** if the player beats the top rung, the Elo estimate **compresses/extrapolates**, so we raised the ladder as the player improved (SF-1700 → 2100 → 2700 → 3000). Absolute numbers carry systematic uncertainty; **relative gains measured on the same ladder are the robust results** (treat absolutes as  $\pm 100$ ). Where two methods are compared, they are always run on the *same* ladder and seeds.

### 2.4 Hardware

- Two Apple-Silicon **Mac Studios (M3 Ultra, 256 GB each)**, MLX framework, 40 Gbps bridge.

---

## 3. Stage 1 — Open Loop (single-pass policy)

---

### 3.1 Topology sweep — architecture beats scale

At fixed moderate data (18M positions) we swept eight architectures:

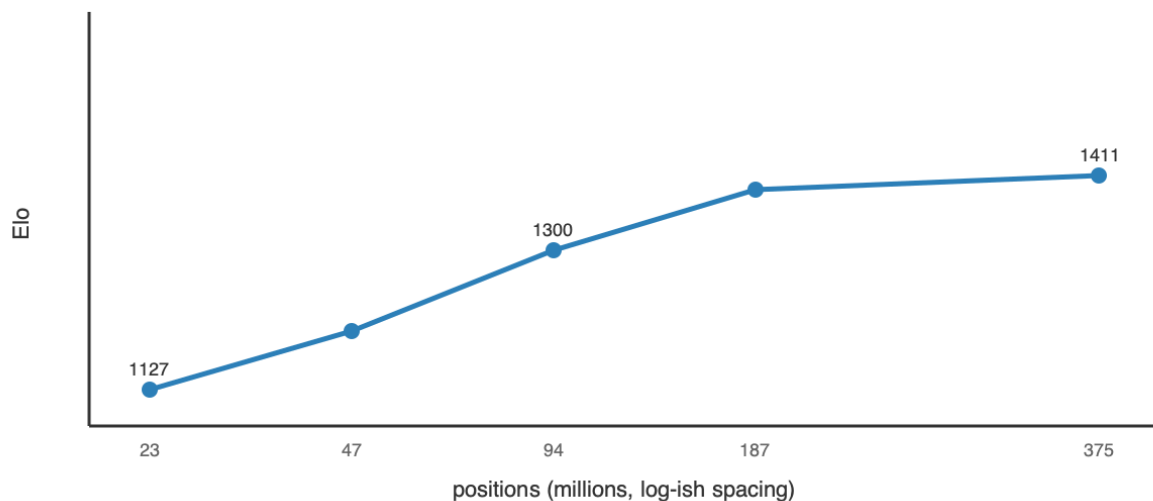
| Topology                     | params   | Elo         | note                          |
|------------------------------|----------|-------------|-------------------------------|
| <b>conv (64×10 / 96×8)</b>   | 2.8–3.4M | <b>1476</b> | best & most efficient         |
| constant-width MLP (1024×6)  | 10.2M    | 1108–1233   | best plain MLP                |
| dual-path (wide+deep, gated) | 2.8M     | 1082        | ties MLP at 3.5× fewer params |
| factored head (from/to)      | 6.2M     | 782         | –450                          |
| funnel (1024→64)             | 1.8M     | 582         | narrowing hurts               |
| pyramid (64→1024)            | 5.0M     | 431         | bad                           |
| bottleneck (512→8)           | 0.5M     | 340         | broken                        |

**Topology conclusion.** Only two shapes are competitive — **constant-width** and **convolution** — and every *taper, factoring, or exotic bottleneck loses badly*. The reason is inductive bias: a taper or bottleneck destroys positional information before the head can use it, while convolution **reuses local tactical patterns across the board via weight sharing**, so it learns a motif once instead of relearning it per square. The practical rule that falls out: **pick the prior that matches the domain's structure (spatial locality) rather than adding raw width**. A 3.45M-param conv matched a 14.4M-param MLP using **22× less data** — architecture, not parameter count, set the strength.

### 3.2 Scaling laws (width and data)

- **Width (MLP, hard):** W<sub>1024</sub> (14.4M) → 1449; W<sub>2048</sub> (47.7M) → 1501: **+52 Elo for 3.3× params** — sharply diminishing. Top-1 accuracy saturates ~0.41 regardless of width.
- **Data (W<sub>1024</sub>):** 1127 / 1207 / 1300 / 1382 / 1411 at 23 / 47 / 94 / 187 / **375M** positions — large early gains, then **+29 on the last doubling** → saturates near the DB's 394M limit.

### Stage 1 — data scaling saturates (MLP W1024, Elo vs positions)



### 3.3 Objective and open-loop ceiling

Soft beats hard by **+94–113 Elo** at subset scale; top-1 saturates while Elo keeps improving via blunder-rate reduction (**top-1 is a misleading metric**). Best open-loop recipe = **conv + soft + full 394M data**, with a **ceiling  $\approx$  2150 Elo**. Cost: 3.45M params, **14 MB**,  **$\sim$ 176 MFLOP /  $\sim$ 1.5 ms per move** — cheap, but capped: no amount of width or data pushed a single pass past  $\sim$ 2150.

## 4. Stage 2 — Closed Loop (search on a value function)

We add a scalar head **Eval(N)**  $\in [0,1]$  = *expected score for the side to move* (  $P(\text{win}) + \frac{1}{2}P(\text{draw})$  ), trained on the eval-DB win-probabilities (held-out **MAE 0.088**, **correlation 0.877** — an excellent value function). Search uses the zero-sum complement identity: our value after a move is  $1 - \text{Eval}(\text{child})$ , so one network plays both sides; terminal nodes return exact 0 / 0.5 / 1.

### 4.1 MCTS vs fixed-depth — the core comparison

We compared two search families on the same value net and ladder:

- **Fixed-depth alpha-beta** (negamax, policy move-ordering, quiescence at leaves):

| depth | raw  | search | gain                                    |
|-------|------|--------|-----------------------------------------|
| 1     | 1661 | 1627   | −34 (1-ply hurts — can't see the reply) |
| 2     | 1685 | 1788   | +103                                    |

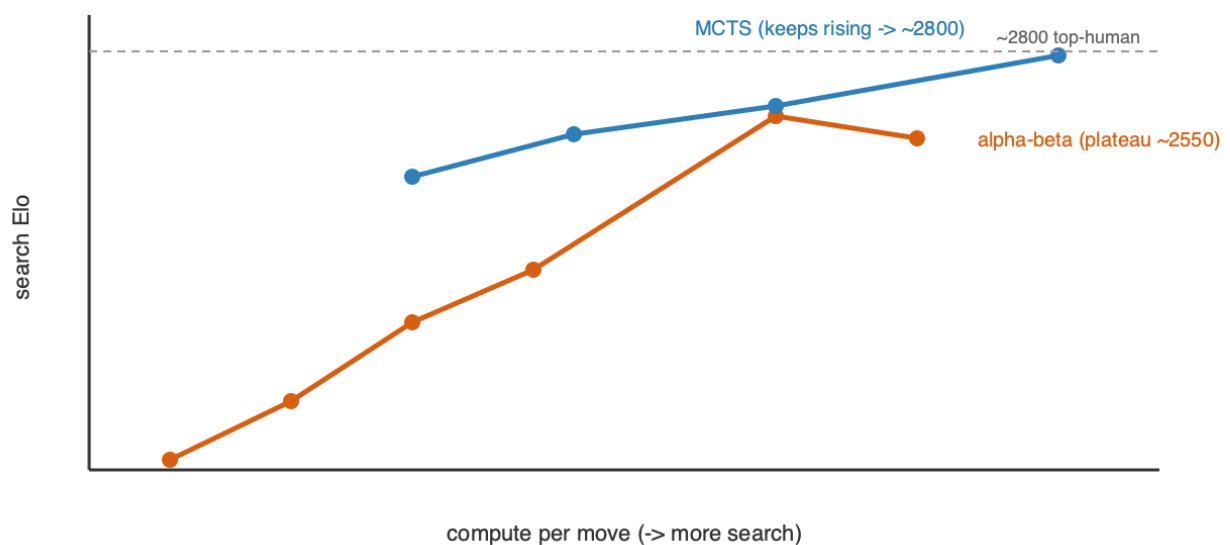
| depth | raw  | search      | gain          |
|-------|------|-------------|---------------|
| 3     | 1895 | 2005        | +110          |
| 4     | 1914 | 2152        | +238          |
| 6     | 2128 | <b>2575</b> | +447          |
| 7     | 2187 | 2513        | plateau ~2550 |

- **Adaptive MCTS/PUCT** (  $Q + c \cdot P \cdot \sqrt{N/(1+n)}$  ), value-head leaves, negamax backup):

| sims       | search Elo                                                         |
|------------|--------------------------------------------------------------------|
| 100        | 2411                                                               |
| 200        | 2530                                                               |
| 400        | 2610 (2661 @ 40 games/rung)                                        |
| <b>800</b> | <b>2749 <math>\approx</math> top-human (<math>\sim</math>2800)</b> |

**Result.** Two clean findings. (i) **1-ply search *hurts* a strong policy** — it commits to a capture without seeing the recapture; depth is where lookahead starts paying. (ii) **MCTS both beats alpha-beta at equal compute and keeps scaling** where fixed depth plateaus ( $\sim$ 2550). Fixed uniform depth spends the same effort on every branch and *amplifies the value net's noise* at the leaves; MCTS instead **allocates search adaptively to sharp lines and averages over that noise**. MCTS is the Stage-2 winner and reaches  $\sim$ 2800 on the fixed 3.45M value net.

### Stage 2 — MCTS out-scales fixed-depth search



## 4.2 Search efficiency — the wide→narrow MCTS cascade (new)

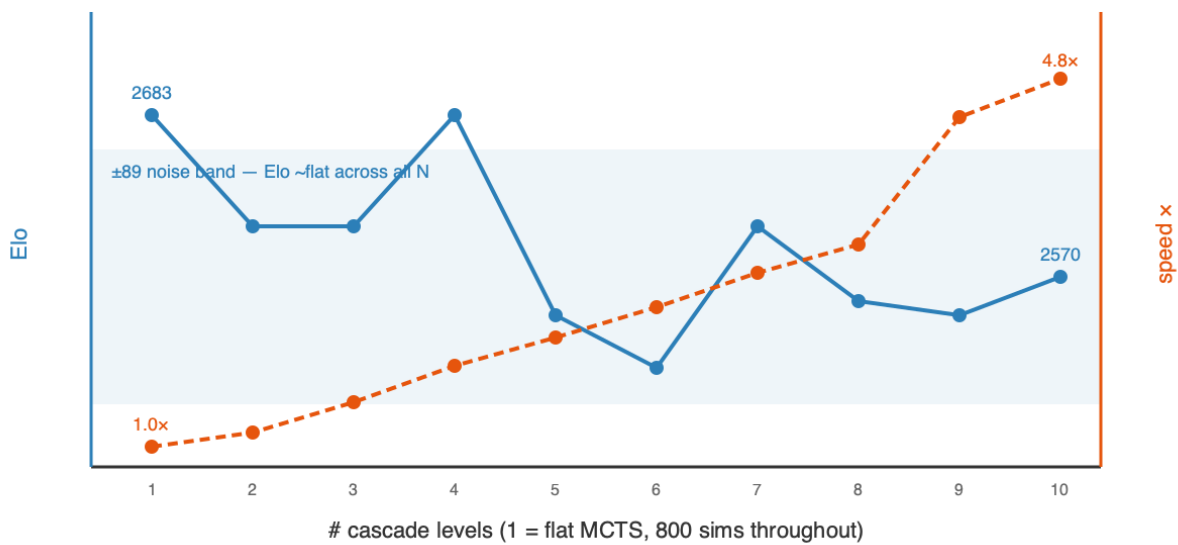
Given that MCTS wins, we asked whether the **allocation** of a fixed simulation budget matters. The **cascade** runs MCTS in *stages* with different knobs — a **wide** stage (all moves, high `c_puct`, few sims) ranks broadly and passes its top-k by visit count to a **narrower, deeper** stage (fewer moves, more sims, lower `c_puct`), funnelling the budget onto the survivors. A shared evaluation cache carries value-net calls forward between stages.

A controlled **N = 1→10 sweep** (one consistent rule generates each funnel; all 800 total sims; same tall ladder, seeds, and openings) gives the full trade-off curve:

| # levels                           | Elo ( $\pm 89$ ) | ms/move | speedup            |
|------------------------------------|------------------|---------|--------------------|
| 1 (flat MCTS)                      | 2683             | 1330    | 1.0×               |
| 3                                  | 2605             | 903     | 1.5×               |
| 6                                  | 2506             | 541     | 2.5×               |
| 9                                  | 2543             | 300     | 4.4×               |
| 10                                 | 2570             | 275     | <b>4.8×</b>        |
| beam-minimax cascade (fixed depth) | 2487             | —       | inferior primitive |

**Result.** Across all ten funnels the Elo stays inside a **single  $\pm 89$  noise band** (range 2506–2683, mean  $\sim 2580$ ) — **statistically flat** — while speed improves **monotonically to 4.8×** at **N = 10**. Funnelling the budget wide→narrow is therefore a **near-pure efficiency win**: it holds flat-MCTS strength while cutting per-move compute up to  $\sim 5\times$ , because the survivors that reach the deep stages are the same moves flat MCTS would have spent its budget on anyway — the funnel just stops paying to search moves it has already ranked out. Any softening at the sharpest funnels is within measurement noise (20 games/rung). The practical dial is simply **more levels = the same strength, cheaper** — turn it up until the noise floor or your latency budget stops you. (The fixed-depth *beam* cascade, by contrast, is a genuinely weaker primitive at 2487 — adaptive MCTS stages matter.)

### Stage 2 — cascade: Elo flat within noise, speed rises to 4.8x



## 4.3 The two-dimensional cost (memory vs GPU-cycles)

|                 | Stage 1 (open)  | Stage 2 (closed)                       |
|-----------------|-----------------|----------------------------------------|
| Memory          | 14 MB           | <b>14 MB — search adds ~0</b>          |
| GPU cycles/move | 1 pass, ~1.5 ms | ~800 passes, ~1 s (or ~0.6 s cascaded) |
| Elo             | ~2150 (capped)  | <b>~2800 (scales with compute)</b>     |

**Stage 1 buys Elo with *memory* and saturates; Stage 2 buys Elo with *GPU cycles* and keeps climbing** — and the cascade shows most of those cycles were waste (up to 4.8× recoverable). The central practical result: **strength is compute, not parameters**.

## 5. Stage 3 — Self-Learning (no external engine, no labels)

**Why this stage is the whole point.** Stages 1–2 leaned on Stockfish labels — a shortcut that *only exists because chess already has a superhuman evaluator and a massive labeled database*. The generic goal is the opposite situation: a **new field where no training data and no existing evaluator exist** — a novel game, an unsolved control or scheduling problem, a scientific-design task. There you *cannot* imitate a teacher because there is none, and you *cannot* score positions with an off-the-shelf engine because none has been built. The only thing you have is the **environment's rules**: which actions are legal, and, eventually, whether you succeeded. Stage 3 deliberately discards every external crutch — no Stockfish, no labels — to test whether strength can be **bootstrapped from self-play and outcomes alone**. This is the transferable question: Stages 1–2 measure what search and capacity buy



when a teacher is available; Stage 3 measures whether the recipe can **stand on its own in a domain where Stages 1–2 are simply impossible** — which is precisely the case for any genuinely new problem.

The loop: the net plays itself with MCTS, trains toward the visit distribution (improved policy) and game result (value), and repeats. The only external input is the rules. We studied four approaches — self-play, a self-referential ladder, a committee, and evolution — and they converge on one story.

### 5.1 Self-play expert iteration (AlphaZero-style)

The net plays itself with MCTS, trains toward the visit distribution (improved policy) and game result (value), and repeats. Two failure modes appeared and were fixed: **cold-start draw-collapse** (a weak net can't force mates, so games drift to draws and the value head gets no signal — fixed with Dirichlet root-exploration noise) and **warm-start forgetting** (training hard on tiny self-play slices overwrote the supervised policy, 2150 → 1407 — fixed with a replay buffer and a gentle learning rate). Stabilized, and even parallelized to 16× throughput (300K replay buffer, eval cache), the raw policy **plateaus ~1950–2030 — below the ~2150 supervised baseline**.

**Negative result (clean).** At two-machine scale, **self-play converges below supervision**. Self-play strength *does* rise with game volume — a real scaling signal — but the *achievable* volume here plateaus under a 394M-supervised net; crossing it needs orders-of-magnitude more games (AlphaZero used ~1000× ours). **Scale is the binding constraint.**



### 5.2 Committee / diversity — teacher-free confidence (new)

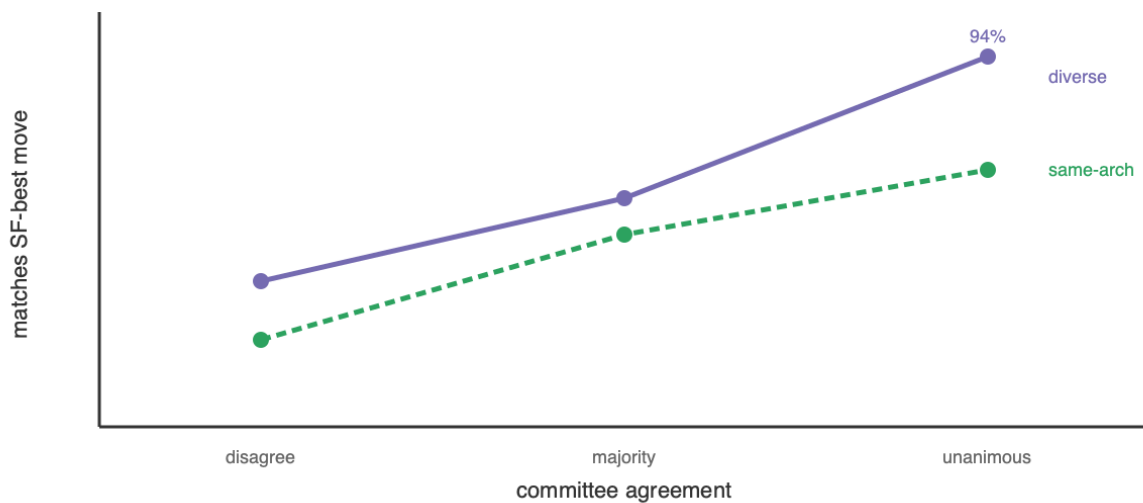
If the bottleneck is the *evaluator*, the cheap way to improve one without training a bigger net is to **ensemble diverse nets**. We tested the hypothesis that **multiple independently-**

**started models either diverge (disagree → uncertain) or converge (agree → likely correct)** — making agreement a self-contained confidence meter with no oracle.

**Validated.** Over 400 positions scored against Stockfish depth-12 (measurement only), agreement strongly predicts correctness:

| members agree | centipawn-loss ↓ | matches SF-best ↑ (same-arch / diverse) |
|---------------|------------------|-----------------------------------------|
| all disagree  | 77.9             | 22% / 37%                               |
| majority      | 40.0             | 49% / 58%                               |
| unanimous     | <b>19.7</b>      | <b>65% / 94%</b>                        |

**Stage 3 — committee agreement predicts correctness**



**But plurality voting does *not* reliably de-bias — a committee-size sweep (3/5/7/9 agents) shows why.** Growing the committee from a correlated conv-soft trio outward:

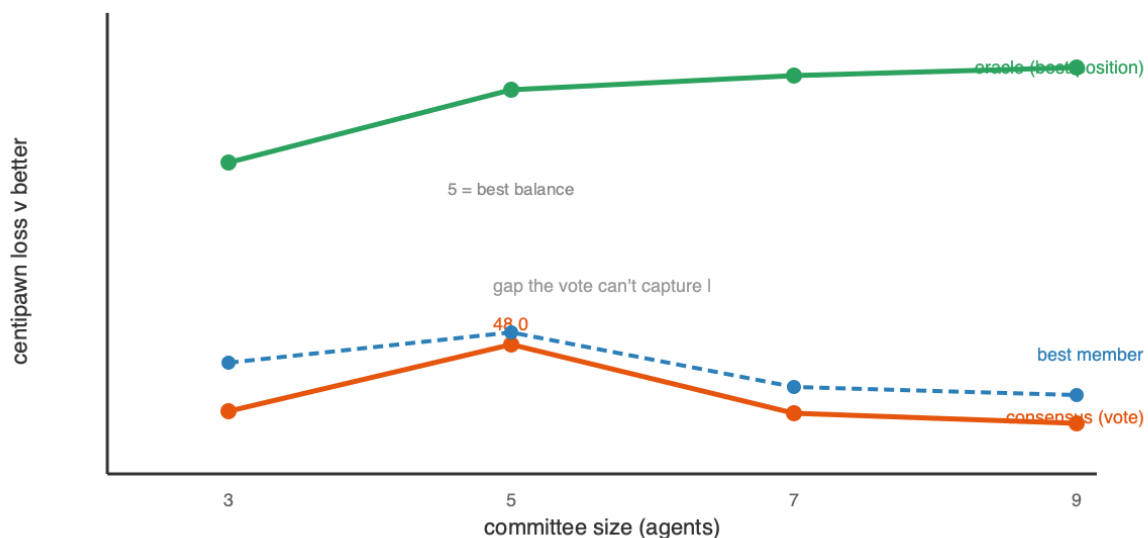
| agents | added views                | consensus CPL | best member | oracle |
|--------|----------------------------|---------------|-------------|--------|
| 3      | conv-soft ×3               | 54.7          | 49.8        | 29.6   |
| 5      | +MLP +hard-objective       | <b>48.0</b>   | 46.8        | 22.2   |
| 7      | +2 conv-soft (data slices) | 54.8          | 52.2        | 20.8   |
| 9      | +MLP +hard                 | 56.0          | 53.1        | 19.9   |

Three findings. (i) **Plurality never clearly beats the best single member** — consensus is slightly *worse* at every size, and the gaps (~1–5 CPL) sit inside the Stockfish-measurement noise (~±3 CPL); an earlier apparent "consensus beats best" was itself within that noise. (ii) **Balance beats count** — the 5-agent committee, where the diverse MLP + hard-objective members are 40% of the vote, is by far the best; *adding correlated members* (the conv-soft data-slice nets at 7 and 9) lets that bloc dominate the plurality and reverts the gain. **More**

**agents  $\neq$  better.** (iii) **The oracle (best member per position,  $\sim 20\text{--}30$  CPL) is  $2\text{--}3\times$  better than the vote** — the diversity *contains* the information, but plurality *cannot extract* it, because it is dominated by the largest correlated bloc.

**Lesson.** The committee gives one thing robustly and one thing not: a **confidence meter** (agreement $\rightarrow$ correctness, validated repeatedly) — but **plurality voting is a weak aggregator** that does not reliably reduce error below the best member. Capturing the large oracle headroom needs a **better aggregator** (soft probability-averaging, or confidence-weighted routing that trusts the per-position agreement) and **balanced**, not merely numerous, diversity. A de-biased ensemble evaluator remains the most promising route to lift the ceiling that search and self-play cannot — but plurality is not it.

### Stage 3 — committee size: plurality never beats the best member; oracle unrealized

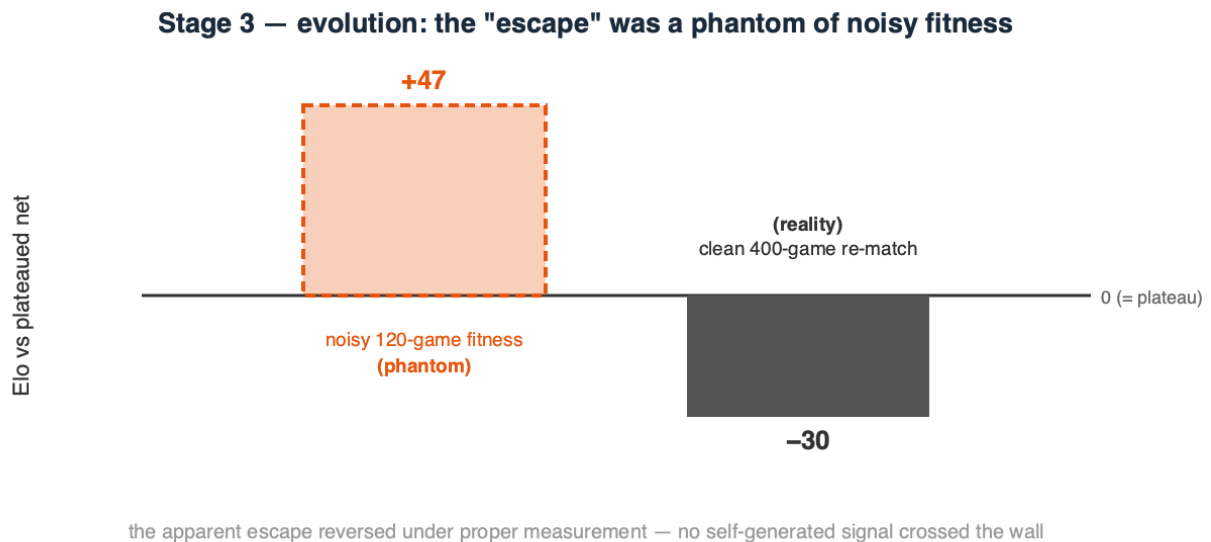


## 5.3 Evolution — mutate / play / score (a plateau-escape attempt)

Gradient self-play optimizes a *proxy* (the net's own biased value targets); we tested whether **derivative-free evolution**, which optimizes the *true* objective — did this mutant win games — could escape the plateau where gradients stalled. From the plateaued net: each generation mutate it into 16 offspring (Gaussian weight noise), play each against a **frozen copy of the plateaued net** (a fixed anchor, so "fitness" is "how well do you beat the plateau"), and crown the best only if it survives a larger confirmation match. A first, naive version selecting against the *moving* champion drifted **downward** — beating your immediate parent is non-transitive in chess and does not imply getting stronger; the fixed anchor fixes that.

**Result: a null result, and a methodological warning.** The run *appeared* to escape — champ-vs- plateau climbed to 0.567 (+47 Elo) and passed a 120-game confirmation. But a **clean 400-game, low-temperature re-match found the "evolved" champion is actually  $-24$  to  $-41$  Elo worse than the plateaued net.** The apparent gain was an artifact of noisy, high-temperature fitness over small samples with best-of-16 selection bias — a **phantom improvement** that vanished under proper measurement. Annealing the

mutation scale up ( $\sigma$  0.03→0.12) found nothing better; larger mutations only degraded the net. So **evolution did not escape the plateau either** — and neither gradient self-play, a self-referential ladder, nor evolution crosses the ~2000 wall. Since the identical net reaches ~2150 under *supervised* labels, the wall is **not capacity** but the ceiling of any *self-generated* signal: a system cannot lift itself past the quality of the signal it produces about itself. (Methodological note for practitioners: relative-fitness selection with small, stochastic samples manufactures phantom gains; only a large, low-variance re-measurement can be trusted — we nearly reported a false escape and caught it only by measuring properly.)



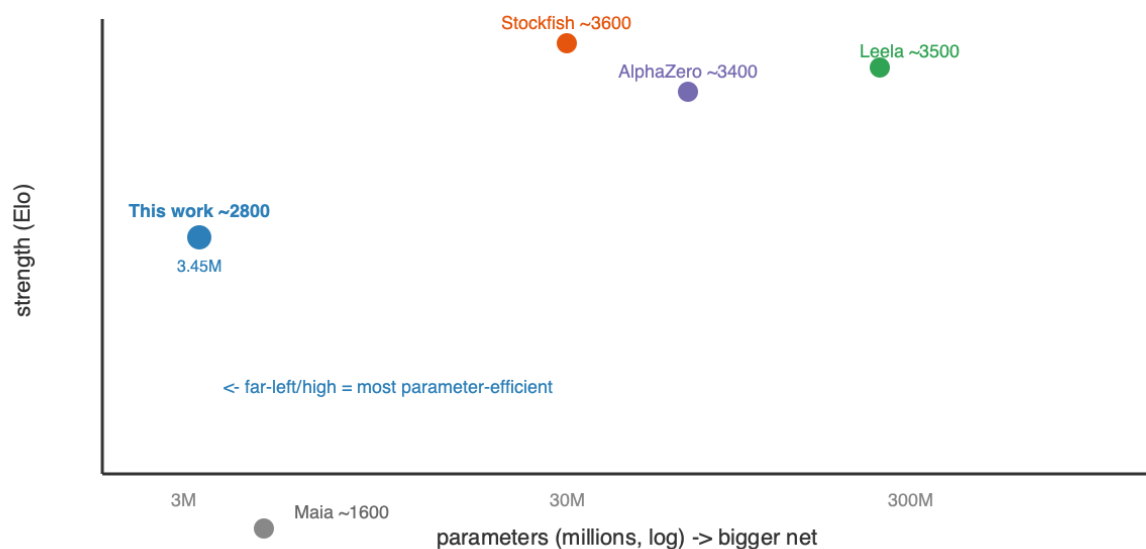
## 6. Comparison to existing systems — parameters, performance, and speed

| System                        | Params (memory)      | Strength (Elo) | Search / move                            | Elo per M-param |
|-------------------------------|----------------------|----------------|------------------------------------------|-----------------|
| <b>This work — raw policy</b> | <b>3.45M</b> (14 MB) | ~2150          | 1 forward pass (~1.5 ms)                 | ~620            |
| <b>This work — + MCTS-800</b> | <b>3.45M</b> (14 MB) | <b>~2800</b>   | 800 net passes (~1.0 s; ~0.6 s cascaded) | <b>~810</b>     |
| Maia (human-like)             | ~few M               | ~1100–1900     | 1 forward pass                           | ~300–500        |
| AlphaZero (chess, 2017)       | ~40–90M              | ~3400+         | 800 MCTS sims (big-net passes)           | ~40–85          |
| Leela Chess Zero (modern)     | ~100–400M+           | ~3500+         | ~1–8k MCTS nodes                         | ~10–35          |

| System           | Params (memory)        | Strength (Elo) | Search / move                  | Elo per M-param |
|------------------|------------------------|----------------|--------------------------------|-----------------|
| Stockfish (NNUE) | ~tens of M (quantized) | ~3600+         | millions of alpha-beta nodes/s | ~100–150        |

**Two axes at once — size and speed.** - **Parameters:** our net is  **$\sim 10\text{--}25\times$  smaller than AlphaZero** and  **$30\text{--}100\times$  smaller than large Leela transformers**, yet reaches  $\sim 2800$  with search. On *Elo-per-million-parameters* it is the extreme efficiency point — an order of magnitude above the big-net engines (which spend their parameters on the last few hundred Elo of a much better value function). - **Speed:** the two paradigms differ. AlphaZero/Leela use a **similar sim count** ( $\sim$ hundreds– thousands) but each sim is a **big-net** forward pass, so their per-move cost is dominated by a  $10\text{--}100\times$  larger network; our sim is a 14 MB pass ( $\sim 1.5$  ms), and the **cascade recovers a further  $\sim 1.6\text{--}4.8\times$** . Stockfish is the opposite regime — a small, heavily-quantized net evaluated at **millions of nodes/second** under alpha-beta. The common structure: **strength = evaluator  $\times$  search; every strong engine is search-heavy, and parameter count is not what separates them.**

Parameter efficiency — strength vs network size (log scale)



**Honest gap.** Top engines sit  $\sim 600\text{--}800$  Elo above  $\sim 2800$ , bought with **far more search and a much better value net** (massive training). Our number is an *efficiency* point — most strength per parameter — not an engine-matching claim, and it carries ladder uncertainty ( $\pm 100$ ).

## 7. Discussion — the unifying conclusion

---

Across all three stages one result recurs: **the ceiling is the evaluator (the network), not the loop around it.**

- **Architecture > parameters.** The right prior (spatial locality + weight sharing) beats raw width; a 14 MB conv reaches strength a 3× larger MLP cannot.
- **Search > scale, at constant memory — but search cannot exceed the net.** MCTS bought +650 Elo (2150→2800) with zero extra parameters and out-scaled fixed depth. Yet flat MCTS and the cascade hit the *same* 2691 wall on the same ladder, because they query the *same* value net. Search reduces the net's *variance* (random error, via averaging many calls) but not its *bias* (systematic blind spots) — and deep uniform search even *amplifies* bias. The cascade's win was therefore **efficiency (up to 4.8× cheaper), not strength.**
- **No self-generated signal crosses the wall — we tried three.** Gradient self-play plateaus below its teacher; a self-referential Elo ladder never promotes; and derivative-free **evolution**, given the fairest shot (unbiased game-outcome fitness, annealed exploration), also fails — its apparent +47-Elo escape was a **noise artifact** that reversed to −30 under a clean 400-game re-match. Since the *same* net reaches ~2150 under supervised labels, the wall is **not capacity** but the ceiling of any signal a system generates about *itself*: external information lifts it, self-generated information cannot.
- **A better evaluator is the only lever — but it is harder than it looks.** The committee gives a robust, teacher-free **confidence signal** (agreement predicts correctness), yet **plurality voting does not reliably de-bias**: across a 3/5/7/9-agent sweep the consensus never clearly beat the best single member, and correlated majority blocs dominate it (balance beats count). The diversity *contains* the information — the per-position oracle is 2–3× better than the vote — but extracting it needs a **better aggregator** (soft-averaging / confidence-routing) and *balanced* diversity, not more agents. Improving the evaluation is the right goal; naive ensembling is not yet the way.
- **Control-theory unification.** Open loop = feedforward; closed loop = MPC/receding-horizon; self-play = iterative learning control — the same three knobs recur in any sequential-decision domain, and in all three the learned *evaluator* is what caps performance.

---

## 8. Limitations and Future Work

---

- Absolute Elo carries systematic uncertainty near the ladder top; relative same-ladder gains are the robust claims.
- Stage 3 is compute-limited (two machines); results characterize the *small-scale* regime.
- Stage-3 fitness/aggregation is noise-limited at our scale: relative-fitness selection with small, stochastic samples produced a **phantom** evolution "escape" that reversed under

a 400-game re-match, and plurality de-biasing gaps sit inside the  $\sim\pm 3$  CPL measurement noise. Larger, lower-variance evaluation is needed to resolve small effects.

- **Next levers, in priority order:** (1) a **better value function** — the true ceiling — via more and better search-labeled data or large-scale self-play; (2) a **better ensemble aggregator** — soft probability-averaging and confidence-weighted *routing* (the per-position oracle is  $2\text{--}3\times$  above plurality), with *balanced* cross-family diversity rather than more agents; (3) **batched/parallel MCTS** (virtual loss) and transposition tables for throughput; (4) endgame tablebases and tuned `c_puct`. All roads converge on the same place: **improve the evaluation, and both the closed-loop and self-play ceilings rise together.**
- 

## 9. Conclusion

---

A 14 MB, 3.45M-parameter network reaches  **$\sim 2800$  Elo by *thinking* (MCTS search), not by *growing* (parameters)**. Adaptive MCTS beats and out-scales fixed-depth search; a wide $\rightarrow$ narrow MCTS cascade matches it at up to  $4.8\times$  less compute. On the self-learning side the results are honestly negative: self-play, a self-referential ladder, and derivative-free evolution all **fail to cross the  $\sim 2000$  plateau** (evolution's apparent escape was a noise artifact), and plurality-voting committees do not reliably de-bias — though model **agreement is a robust teacher-free confidence signal**. The transferable result, proven from every direction we pushed, is that **the learned evaluator is the bottleneck** — search redistributes its knowledge and self-generated signal cannot exceed its own quality; only a better evaluator (better labels, more scale, or a better *aggregator* than plurality) raises the ceiling. A quantified recipe, and an honest map of its limits, for compact, search-driven sequential decision-making in general.

---

## Appendix — Implementation

---

- `chessnet/model.py` — conv/MLP/dual-path + value head. `chessnet/search.py` — alpha-beta, MCTS/PUCT, quiescence, the wide $\rightarrow$ narrow cascade (`MultiStageMCTSPlayer`). `chessnet/committee.py` — ensemble inference + agreement signal. `chessnet/train.py` — soft/hard + value training. `scripts/selfplay.py` — self-play iteration.
- **Best model:** `runs/conv_value_llm1` (conv-96 $\times$ 8 + value, 3.45M params).
- **Key hyperparameters:** conv width 96, depth 8; lr  $5e-4$  (train) /  $1e-4$  (self-play); gradient clip 1.0; MCTS `c_puct` 1.5; Dirichlet  $\alpha$  0.3; replay buffer 120K–300K.